Tabla de contenidos

1. 1. Excepciones
2. Tipos de excepciones
    a. `SyntaxError`
    b. `IndentationError`
    c. `NameError`
    d. `ZeroDivisionError`
    e. `IndexError`
    f. `KeyError`
    g. `AttributeError`
    h. `TypeError`
    i. `ValueError`

1. Excepciones

En Ciencia de la Computación, una **excepción** es una condición anómala o inesperada que ocurre durante la ejecución de un programa y que interrumpe su **ejecución esperada**. Cuando esto sucede, los sistemas generan un evento denominado **excepción** que altera el curso habitual del programa.

Entre las situaciones más comunes que pueden generar excepciones se encuentran:

- Operaciones no definidas (ej. división por cero).
- Acceso a regiones de memoria no permitidas (ej. leer más allá de los límites de una lista).
- Acceder a una *key* inexistente en un diccionario.
- Construir un número a partir de un *string* con formato incorrecto: `int("34C")`.
- Usar una variable no definida.
- Invocar un método inexistente en un objeto.
- Realizar una operación prohibida para un tipo de dato (ej. modificar una tupla).

Aunque estos casos podrían manejarse con estructuras de control como `if`/`elif`/`else`, hacerlo así complica el código y lo vuelve más difícil de mantener. Además, cada nueva condición obligaría a modificar múltiples partes del programa, aumentando el riesgo de errores.

Por ello, lenguajes como Python permiten **generar** (*raise*) excepciones cuando ocurre una situación anómala y **capturarlas** (*catch*) en bloques de código diseñados para tratarlas. Este proceso, llamado **manejo de excepciones** (*exception handling*), permite:

- Corregir el problema.
- Notificar al usuario.
- Ignorarlo y continuar.
- O tomar cualquier acción que permita retomar la ejecución normal del programa.

Si una excepción no es manejada, se reporta al sistema operativo y, por lo general, el programa finaliza abruptamente. En Python, esto significa que el intérprete detendrá la ejecución y mostrará un mensaje indicando el tipo de error ocurrido.

En Python, las excepciones son objetos que heredan de la clase Exception y se crean en el momento en que se detecta el problema. Algunas excepciones en Python 3.x pueden deberse a:

- Error del usuario al ingresar algún dato.
- Argumentos no válidos al llamar una función (cantidad de parámetros, tipo de datos, etc.).
- Error de sintaxis, al intentar ejecutar código con sintaxis ambigua o incorrecta.
- Intentar utilizar variables inexistentes (solo se genera cuando se accede a ellas).
- Invocar un método inexistente dentro de un objeto.
- Ejecutar una operación prohibida para un tipo de dato (por ejemplo, modificar un elemento de una tupla).

Dado que Python es un lenguaje **interpretado**, estas **excepciones se producen en tiempo de ejecución (runtime exceptions)**. Esto implica que un error como el uso de una variable no definida no provocará una excepción a menos que el flujo del programa llegue a esa instrucción.

En cambio, los lenguajes **compilados** (como Java, C o C#) son traducidos a código máquina antes de ejecutarse. Durante la compilación, pueden detectarse errores como variables no definidas, uso incorrecto de tipos o métodos inexistentes. Sin embargo, las excepciones propias de la ejecución seguirán ocurriendo únicamente al momento de correr el programa.

Un **manejo adecuado de excepciones** es clave para desarrollar programas **robustos**, capaces de recuperarse ante errores inesperados o entradas inválidas. Después de todo, no nos gustaría que nuestro juego favorito se cayera por completo cada vez que nos equivocamos y escribimos una `O` en un lugar de un `0` (o una `l` en lugar de `1`).

Tipos de excepciones

Cada lenguaje posee distintos tipos de excepciones. A continuación revisaremos algunos ejemplos de clases de excepciones comunes en Python. Todas ellas son clases que heredan de `Exception`.

SyntaxError

Se genera cuando una sentencia del programa está mal escrita y viola sus reglas sintácticas. Esta excepción se levanta antes de comenzar a ejecutar el programa, se levanta al leer todo el código que se piensa ejecutar.

Por ejemplo, el uso de la sentencia `print` sin paréntesis es válido en Python 2.x, pero incorrecto en Python3.x.

```
print "Hello World"
```

```
  Cell In[2], line 1
    print "Hello World"
    ^
SyntaxError: Missing parentheses in call to 'print'. Did you mean
print(...)?
```

En cambio, el uso de `print` válido en Python3.x. es como una función y requiere parámetros dentro de paréntesis `()`. En Python2.x `print` es una palabra clave reservada, pero no una función.

```
print("Hello World")
```

```
Hello World
```

Cuando se nos olvida cerrar un paréntesis, un string, o escribir un `:` también es un error de sintaxis.

```
print("Uwu"
```

```
  Cell In[4], line 1
    print("Uwu"
                ^
SyntaxError: incomplete input
```

```
"Uwu
```

```
  Cell In[5], line 1
    "Uwu
    ^
SyntaxError: unterminated string literal (detected at line 1)
```

```
for i in range(9)
```

```
  Cell In[6], line 1
    for i in range(9)
                      ^
SyntaxError: expected ':'
```

IndentationError

Este error, que hereda de `SyntaxError`, ocurre cuando una línea de código no respeta la indentación correcta. En Python, cada vez que se define un **bloque de código** o *scope* (por ejemplo, con `if`, `for`, `def`, etc.) es obligatorio incluir al menos una instrucción correctamente indentada dentro de él. En el siguiente ejemplo, se abre un bloque con `if`, pero la línea posterior no está indentada, lo que provoca la excepción.

```
user = "Alice"
if user != "Queen of Hearts":
    # Verifica si el user es "Queen of Hearts"
print("Watch out! I am the Queen of Hearts from Wonderland")
```

```
  Cell In[28], line 4
    print("Watch out! I am the Queen of Hearts from Wonderland")
    ^
IndentationError: expected an indented block after 'if' statement on
line 2
```

Cabe destacar que un comentario no evita este error porque no es código ejecutable. En cambio, una instrucción como `pass` sí lo hace.

```
user = "Alice"
if user != "Queen of Hearts":
    # Este comentario no sirve
    pass  # El pass
print("Watch out! I am the Queen of Hearts from Wonderland")
```

```
Watch out! I am the Queen of Hearts from Wonderland
```

NameError

Este error ocurre cuando el programa intenta acceder a un nombre —ya sea de una variable, función o clase— que no ha sido definido previamente en el ámbito local o global. En otras palabras, el identificador es desconocido para el intérprete.

Por ejemplo, en Python 2.x la función para leer datos del usuario es distinta a la de Python 3.x, por lo que usar la versión incorrecta del comando puede generar un `NameError`.

```
a = raw_input("Enter a number: ")
a
```

```
---------------------------------------------------------------------
-----
NameError                                 Traceback (most recent call
last)
Cell In[9], line 1
----> 1 a = raw_input("Enter a number: ")
      2 a

NameError: name 'raw_input' is not defined
```

En Python3.x. la función `raw_input()` fue renombrada a `input()`, por lo tanto, ante una llamada este primer nombre no puede ser hallado por el intérprete de Python.

```
a = raw_input("Enter a number: ")
a
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[11], line 1
----> 1 a = raw_input("Enter a number: ")
      2 a

NameError: name 'raw_input' is not defined
```

Cuando se intenta leer el valor de una variable que no ha sido previamente definida, también ocurre un error de nombre.

```
a = 42
b = c + a
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[12], line 2
      1 a = 42
----> 2 b = c + a

NameError: name 'c' is not defined
```

Recordemos que las excepciones en Python solo se detectan en tiempo de ejecución (*runtime*), ya que es un lenguaje interpretado. Observa el siguiente código:

```
a = 42
if a < 42:
    b = c + a
else:
    b = a * 2
print(b)
```

```
84
```

Este ejemplo contiene un error, al igual que el código anterior, ya que no se ha definido la variable `c`. Sin embargo, si esa instrucción nunca se ejecuta, la excepción no se genera. Si hubiésemos escrito un código similar en un lenguaje compilado, el proceso de compilación sí hubiese detectado este error (y lo hubiésemos corregido).

ZeroDivisionError

Esta excepción es lanzada cuando el segundo elemento, o denominador, de una división es cero.

En el ejemplo vemos que la función `divide` está correctamente escrita, sin embargo, los valores ingresados por el usuario producen este error matemático.

```
def divide(x, y):
    return x / y


r = 4
w = divide(5, r)
print(w)
z = divide(5, r - 4)
print(z)
```

```
1.25

---------------------------------------------------------------------------
-----
ZeroDivisionError                         Traceback (most recent call
last)
Cell In[14], line 8
      6 w = divide(5, r)
      7 print(w)
----> 8 z = divide(5, r - 4)
      9 print(z)

Cell In[14], line 2, in divide(x, y)
      1 def divide(x, y):
----> 2     return x / y

ZeroDivisionError: division by zero
```

En este caso, el problema no está en la definición de la función `divide` (aunque la excepción se genere ahí), sino en el **uso** que se hace de ella y en los argumentos que recibe. En el primer llamado, los parámetros son válidos; en el segundo, en cambio, se pasa un valor que provoca la excepción (¿podríamos haberlo previsto antes de llamar a `divide`?).

Este tipo de error solo puede detectarse en **tiempo de ejecución**, ya que al leer el código de `divide` no es posible anticipar si alguna vez será llamada con un `y` igual a `0`.

La función no es robusta, es decir, no está preparada para resistir este tipo de fallos. Más adelante, al aprender a detectar y manejar excepciones, podremos implementar mecanismos para corregirlo.

IndexError

Este error se produce cuando se intenta acceder a un índice que está fuera del rango válido de una secuencia.

El caso más común ocurre al referirse a un elemento de una lista, tupla u otra estructura indexable con un número de índice que excede el límite permitido por la cantidad de elementos

que contiene. En Python, las secuencias como las listas se indexan desde `0` hasta `len(lista) - 1`.

```
age = (36, 23, 12)
age[3]
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[15], line 2
      1 age = (36, 23, 12)
----> 2 age[3]

IndexError: tuple index out of range
```

KeyError

Esta excepción alerta sobre el uso incorrecto o inválido de llaves (*keys*) en diccionarios y *mappings*, similarmente a `IndexError` en listas.

En el ejemplo a continuación, el usuario pide un dato asociado a una llave inexistente en el diccionario. Al no existir, se levanta la excepción (aunque podríamos haber usado `defaultdict` y no toparnos con esta excepción).

```
books = {"author": "Juanito Los Palotes",
         "pages": 9877}

books["publisher"]
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[16], line 4
      1 books = {"author": "Juanito Los Palotes",
      2          "pages": 9877}
----> 4 books["publisher"]

KeyError: 'publisher'
```

AttributeError

Esta excepción alerta sobre el uso incorrecto de métodos o atributos de una clase o tipo de dato.

En este ejemplo, la clase `Car` ha definido el método avanzar. Sin embargo, el usuario ejecuta el método `stop()` que no existe en ella.

```
class Car:

    def __init__(self, doors=4):
        self.doors = doors

    def move_forward(self):
        print("moving forward")


chevy = Car()
chevy.stop()
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call
last)
Cell In[17], line 11
      7         print("moving forward")
     10 chevy = Car()
---> 11 chevy.stop()

AttributeError: 'Car' object has no attribute 'stop'
```

TypeError

Esta excepción indica que hubo un manejo erróneo de **tipos** de datos. Es decir, cuando se intenta ejecutar una operación o función con un argumento que no pertenece al tipo correcto para la ejecución.

Un error común es concatenar una lista con algo que no es del tipo `list`. En este ejemplo, la función está definida como la suma de variables, sin embargo, debido a *duck-typing*, el operador `+` se comporta distinto de acuerdo a los tipos de los datos que recibe.

```
def combine(x, y):
    return x + y


ages = [36, 23, 12]
more_ages = [40, 25]

print(combine(ages[1], more_ages[0]))  # 23 + 40 = 63
print(combine(ages, more_ages))        # concatenates the two lists
print(combine(2, ages))                # TypeError: unsupported
operand types for +: 'int' and 'list'
```

```
63
[36, 23, 12, 40, 25]
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call
last)
Cell In[18], line 9
      7 print(combine(ages[1], more_ages[0]))  # 23 + 40 = 63
      8 print(combine(ages, more_ages))        # concatenates the two lists
----> 9 print(combine(2, ages))                # TypeError: unsupported
     10 operand types for +: 'int' and 'list'

Cell In[18], line 2, in combine(x, y)
      1 def combine(x, y):
----> 2     return x + y

TypeError: unsupported operand type(s) for +: 'int' and 'list'
```

```
TypeError                                 Traceback (most recent call
last)
Cell In[18], line 10
      8 print(combine(ages[1], more_ages[0]))  # 23 + 40 = 63
      9 print(combine(ages, more_ages))        # concatenates the two
lists
---> 10 print(combine(2, ages))                # TypeError:
unsupported operand types for +: 'int' and 'list'

Cell In[18], line 2, in combine(x, y)
      1 def combine(x, y):
----> 2     return x + y

TypeError: unsupported operand type(s) for +: 'int' and 'list'
```

Podemos observar que en el primer llamado el operador `+` recibió dos `int`, por lo tanto, se comportó como una suma aritmética. En el segundo llamado, el operador `+` recibió dos `list`, por lo tanto, se comportó como una concatenación de listas. Sin embargo, en el tercer caso recibió `int` y `list`, y no estaba definido qué hacer con ese tipo de operandos.

Otro error suele surgir cuando intentamos llamar un objeto que no tiene implementado el método `__call__`. Este método permite que el objeto pueda ser llamado cuando utilizamos los parentesis `()` (también nombrado como operador de llamada). Por ejemplo, esto ocurre cuando colocamos paréntesis después de un `int`, `str`, `list`, entre otros.

```
name = "Anya"
name()

---------------------------------------------------------------------
-----
TypeError                                 Traceback (most recent call
last)
Cell In[19], line 2
      1 name = "Anya"
----> 2 name()

TypeError: 'str' object is not callable

amount = 42
amount()

---------------------------------------------------------------------
-----
TypeError                                 Traceback (most recent call
last)
Cell In[20], line 2
      1 amount = 42
----> 2 amount()

TypeError: 'int' object is not callable
```

```
books = ["Don Quijote de la Mancha", "Los cuentos de Canterbury", "El Principito", "El retrato de Dorian Gray"]
books()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[21], line 2
      1 books = ["Don Quijote de la Mancha", "Los cuentos de Canterbury", "El Principito", "El retrato de Dorian Gray"]
----> 2 books()

TypeError: 'list' object is not callable
```

ValueError

Esta excepción indica que hubo un manejo erróneo de **valor** de datos. Es decir, cuando se intenta ejecutar una operación o función con un argumento cuyo **valor no era apropiado** para la ejecución esperada.

Podemos encontrar este tipo de error en múltiples funciones conocidas. Por ejemplo, los *strings* de Python tienen el método `split` que permite separar la cadena de texto según algún separador que recibe como argumento:

```
"My string separable by spaces".split(" ")
```

```
['My', 'string', 'separable', 'by', 'spaces']
```

```
"My string separable by spaces".split("")
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[23], line 1
----> 1 "My string separable by spaces".split("")

ValueError: empty separator
```

Si a esta función le entregamos como argumento un *string* vacío, arroja `ValueError` debido a que necesita que este tenga contenido para lograr separar una palabra.

Otro ejemplo es el método `remove` de listas de Python, este recibe un elemento que remover de la lista y de encontrarlo, lo hace:

```
my_list = [1, 2, 3, 4, 5]
my_list.remove(3)
my_list
```

```
[1, 2, 4, 5]

my_list = [1, 2, 3, 4, 5]
my_list.remove(6)
my_list

---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[25], line 2
      1 my_list = [1, 2, 3, 4, 5]
----> 2 my_list.remove(6)
      3 my_list

ValueError: list.remove(x): x not in list
```

En el último caso, si se le entrega un elemento que no existe en la lista, arroja `ValueError` debido a que necesita que el elemento esté en la lista para removerlo. A diferencia del ejemplo anterior, el error no era intrínseco del valor del argumento, sino que era específico a la lista que llamó el método. Al llamar `remove(6)` de una lista que si contiene un `6`, no hubiera arrojado un error. Para `remove` el valor no era apropiado para la instancia que llamaba al método, mientras que el ejemplo de `split` el valor no era apropiado para ningún caso de llamado.

Es importante notar la sutil diferencia entre `ValueError` y `TypeError`. De cierta forma, todo `TypeError` es un `ValueError`, ya que detecta problemas del tipo de un argumento, que inherentemente habla del valor recibido, pero no todo `ValueError` es un `TypeError`. Prueba de esto último fueron los últimos dos ejemplos, donde los argumentos tenían un tipo apropiado para el contexto, pero no el valor.

Pero si todo `TypeError` es un `ValueError`, ¿por qué definir y usar ambos? La respuesta radica a que es más conveniente siempre tener la mayor información posible sobre las excepciones que se lancen. `TypeError` es un caso específico de `ValueError`, por lo tanto, aporta más información sobre el problema que ocurrió en la ejecución del código y ayuda más a manejarlo después. `ValueError` es un error bastante genérico, y ocurre cada vez que se **espera** que el *input* recibido cumpla cierta propiedad y no lo hace.